

UNIT TESTING IMPLEMENTATION REPORT

Flat Rental System (LuxeFlats)

Using JUnit 5 and Mockito

Project: Flat Rental & Booking Marketplace (LuxeFlats)

Testing Framework: JUnit 5 (Jupiter)

Mocking Framework: Mockito

Programming Language: Java 21

Backend Framework: Spring Boot

Build Tool: Maven

IDE: VS Code / Eclipse / IntelliJ IDEA

Team:

[Your Name / Team Member Names & IDs]

Introduction

Unit testing was implemented for the Flat Rental System (LuxeFlats) backend to verify that individual services, domain logic, and validation methods operate correctly and reliably in complete isolation. JUnit 5 was utilized as the standard Java testing framework to structure and execute test suites, while Mockito was employed to mock external dependencies such as repositories, security utilities, and inter-service HTTP clients without requiring active MySQL database or network connectivity.

The test suite covers successful core workflows, input validations, exception handling, role-based authorization, state transition guards, and payment processing. A total of **46 test cases** were executed across all microservices, and **all 46 test cases passed successfully (100% success rate)**.

Services Tested

1> Authentication & User Service

The Authentication Service handles user registration, role parsing, BCrypt password hashing, authentication, JWT token generation & claim parsing, and contact detail updates.

Test Classes: AuthServiceTest, JwtUtilTest

Total test cases: 15

S. No.	Test Case	Method Tested	Explanation	Result
1	Register new user	register()	Verifies that a new user (tenant/owner) is registered with BCrypt encrypted password and correct role.	Pass
2	Register duplicate username	register()	Verifies that DuplicateResourceException is thrown when the requested username is already taken.	Pass
3	Register duplicate email	register()	Verifies that DuplicateResourceException is thrown when the email address is already registered.	Pass
4	Register invalid role	register()	Verifies that IllegalArgumentException is thrown when an unsupported user role is supplied.	Pass
5	Login successfully	login()	Verifies that valid credentials return authenticated user details and a signed JWT access token.	Pass
6	Login invalid username	login()	Verifies that InvalidCredentialsException is thrown when the username does not exist.	Pass
7	Login wrong password	login()	Verifies that InvalidCredentialsException is thrown when an incorrect password is provided.	Pass
8	Get user by valid ID	getUserById()	Verifies that user profile information is correctly returned when looking up an existing user ID.	Pass
9	Get user by invalid ID	getUserById()	Verifies that ResourceNotFoundException is thrown when the requested user ID does not exist.	Pass
10	Update user contact info	updateUserContact()	Verifies that phone number, email, and preferred contact channel are updated and persisted.	Pass

S. No.	Test Case	Method Tested	Explanation	Result
11	Get all users	getAllUsers()	Verifies that the service retrieves the list of all registered users in the database.	Pass
12	Generate and extract token claims	generateToken() / extractClaims()	Verifies JWT token issuance and proper extraction of username, user ID, and role claims.	Pass
13	Validate matching token	isTokenValid()	Verifies that isTokenValid returns true when the token subject matches the authenticated username.	Pass
14	Reject non-matching token	isTokenValid()	Verifies that isTokenValid returns false when username does not match token subject.	Pass
15	Reject expired token	isTokenValid() / extractClaim()	Verifies that expired tokens are detected and rejected by throwing ExpiredJwtException.	Pass

Result: 15/15 Passed (100%)

2> Property Service

The Property Service manages rental flat listings, specifications, rent & deposit pricing, geographic search, and owner modification authorization.

Test Class: **PropertyServiceTest**

Total test cases: 10

S. No.	Test Case	Method Tested	Explanation	Result
1	Create property listing	createProperty()	Verifies that an owner can successfully list and create a new rental property with full specifications.	Pass
2	Get all properties	getAllProperties()	Verifies that all available property listings in the system are retrieved properly.	Pass
3	Get property by valid ID	getPropertyById()	Verifies that property details and metadata are returned when querying a valid property ID.	Pass
4	Property not found	getPropertyById()	Verifies that ResourceNotFoundException is thrown when querying a non-existing property ID.	Pass
5	Filter properties by city	getPropertiesByCity()	Verifies that properties belonging to a specific target city (case-insensitive) are retrieved.	Pass
6	Get properties by owner	getPropertiesByOwner()	Verifies that all properties registered by a specific owner ID are returned for owner dashboard view.	Pass
7	Update property by owner	updateProperty()	Verifies that an authorized owner can modify property title, rent amount, and details.	Pass
8	Update property by non-owner	updateProperty()	Verifies that UnauthorizedActionException is thrown when a non-owner attempts to edit a property.	Pass
9	Delete property by owner	deleteProperty()	Verifies that the rightful owner can delete their property listing from the platform.	Pass
10	Delete property by non-owner	deleteProperty()	Verifies that UnauthorizedActionException is thrown when an unauthorized user attempts to delete a property.	Pass

Result: 10/10 Passed (100%)

3> Booking Service

The Booking Service orchestrates tenant booking applications, date validations, state machine transitions, tenant verification, cancellation reasons, and owner approval.

Test Class: BookingServiceTest

Total test cases: 11

S. No.	Test Case	Method Tested	Explanation	Result
1	Create booking draft	createBooking()	Verifies that a tenant can initiate an application with default status APPLICATION_DRAFT.	Pass
2	Booking with token reference	createBooking()	Verifies that booking status promotes to PENDING_OWNER_APPROVAL when token payment reference is provided.	Pass
3	Invalid booking dates	createBooking()	Verifies that InvalidBookingException is thrown when the end date is on or before the start date.	Pass
4	Get booking by valid ID	getBookingById()	Verifies that booking records are retrieved accurately when searching by existing booking ID.	Pass
5	Booking not found	getBookingById()	Verifies that ResourceNotFoundException is thrown when querying a non-existing booking ID.	Pass
6	Valid status transition	updateStatus()	Verifies that valid state promotion from APPLICATION_DRAFT to PENDING_OWNER_APPROVAL succeeds.	Pass
7	Modify terminal status	updateStatus()	Verifies that InvalidBookingException is thrown when attempting to modify an already CANCELLED booking.	Pass
8	Cancel booking with reason	cancelBooking()	Verifies that booking transitions to CANCELLED and tenant cancellation reason is saved in database.	Pass
9	Salary validation	submitVerification()	Verifies that tenant verification is rejected with InvalidBookingException when salary is zero or negative.	Pass
10	Approve with contact share	approveWithContactShare()	Verifies that the owner can approve booking, change status to CONFIRMED,	Pass

S. No.	Test Case	Method Tested	Explanation	Result
			and record contact details.	
11	Unauthorized approval attempt	approveWithContactShare()	Verifies that non-owners cannot approve a booking application or share owner contact details.	Pass

Result: 11/11 Passed (100%)

4> Payment Service

The Payment Service handles token deposits, rent payments, duplicate transaction checks by payment type, transaction reference generation, and refund operations.

Test Class: `MockPaymentServiceTest`

Total test cases: 10

S. No.	Test Case	Method Tested	Explanation	Result
1	Create token payment	<code>createPayment()</code>	Verifies that token payment is created with status COMPLETED and unique transaction reference (LF-TXN-).	Pass
2	Duplicate token payment guard	<code>createPayment()</code>	Verifies that <code>PaymentProcessingException</code> is thrown if another TOKEN payment is attempted for the same booking.	Pass
3	Rent payment after token success	<code>createPayment()</code>	Verifies that RENT payment is allowed even after TOKEN payment succeeded (regression fix validation).	Pass
4	Get payment by valid ID	<code>getPaymentById()</code>	Verifies that payment record details are returned accurately when querying a valid payment ID.	Pass
5	Payment not found	<code>getPaymentById()</code>	Verifies that <code>ResourceNotFoundException</code> is thrown when searching for a non-existing payment ID.	Pass
6	Get payments by booking ID	<code>getPaymentsByBooking()</code>	Verifies that all payment records (token + rent) associated with a specific booking are retrieved.	Pass
7	Get payments by tenant ID	<code>getPaymentsByTenant()</code>	Verifies that all payment history records for a specific tenant ID are returned.	Pass
8	Update payment status	<code>updateStatus()</code>	Verifies that payment status can be updated (e.g. from COMPLETED to REFUNDED) upon refund approval.	Pass
9	Initiate refund for completed payment	<code>initiateRefund()</code>	Verifies that refund can be initiated for a completed payment and payment status transitions to REFUNDED.	Pass
10	Reject refund for pending payment	<code>initiateRefund()</code>	Verifies that <code>PaymentProcessingException</code> is thrown when attempting to refund a non-completed payment.	Pass

Result: 10/10 Passed (100%)

Overall Test Result

Service	Test Cases	Passed	Failed	Success Rate
Authentication & User Service	15	15	0	100%
Property Service	10	10	0	100%
Booking Service	11	11	0	100%
Payment Service	10	10	0	100%
Total	46	46	0	100%

Conclusion

The unit testing implementation was successfully completed for all core backend microservices of the Flat Rental System (LuxeFlats). A total of **46 test cases** were executed, and **all 46 passed** with zero failures and zero errors.

The tests validate positive operational paths as well as negative edge cases, including duplicate resource rejection, role-based access security, invalid dates and salary ranges, state machine terminal restrictions, independent token/rent transaction types, and refund flows. Using Mockito dynamic proxies ensured rapid, isolated execution with 100% reproducibility across environments.